Universidad Nacional
Vicerrectoría de Docencia
Programa Éxito Académico

Matrices de objetos, herencia y downcasting

Colecciones con matrices automáticas y dinámicas
Guía teórica de repaso para la tutoría

EIF201 Programación I · II Ciclo 2026 · Semana 14 · Sesión del lunes 19 de octubre
Andrey Jesús Solano Rojas, tutor

1. Propósito y alcance

La semana 14 del cronograma continúa con las colecciones basadas en matrices, automáticas y dinámicas. Esta guía lleva el tema hasta el caso que aparece en los proyectos y exámenes del curso: una **matriz dinámica de punteros a objetos** encapsulada en una clase. Además, integra la **herencia** (semana 9) para que la matriz guarde objetos de distintas clases derivadas, y presenta el **downcasting** con `dynamic_cast`, que se profundiza en la semana 15 junto con el polimorfismo.

Al finalizar el repaso, el estudiante debería ser capaz de:

- recorrer una matriz por filas, por columnas y por diagonales;
- crear y liberar una matriz dinámica de tipos primitivos y de punteros a objetos;
- implementar una clase que use una matriz como colección, con validación de posiciones;
- retornar una posición encontrada mediante parámetros por referencia;
- almacenar objetos de una jerarquía en la matriz y operar sobre ellos con métodos virtuales;
- distinguir *upcasting* de *downcasting* y usar `dynamic_cast` de forma segura.

Ejemplo conductor. Una sala de cine con filas y columnas de asientos. Cada asiento está libre (`nullptr`) o apunta a una reserva, que puede ser general, de estudiante o VIP. El código completo está en `sala_cine.cpp`.

2. Matrices automáticas

Antes de las matrices dinámicas conviene repasar las automáticas: su sintaxis de acceso es la misma, y sus limitaciones explican por qué se necesitan las dinámicas.

2.1 Declaración y almacenamiento

Una matriz automática tiene sus dimensiones fijas al compilar y vive en la pila, igual que un arreglo:

```
int m[3][4] = {{1, 2, 3, 4},
               {5, 6, 7, 8},
               {9, 10, 11, 12}};     // m[fila][columna]
```

En memoria, C++ guarda la matriz **por filas**: primero toda la fila 0, después toda la fila 1, y así sucesivamente, en un solo bloque contiguo.



Figura 1. La matriz de 3 × 4 y su almacenamiento en memoria.

2.2 Recorridos

Casi todas las operaciones sobre una matriz son un recorrido. Lo que cambia es cuál índice queda fijo y cuál avanza:

Recorrido	Patrón de índices	Uso típico
Completo por filas	`for f { for c { m[f][c] } }`	Mostrar, sumar, buscar
Completo por columnas	`for c { for f { m[f][c] } }`	Totales por columna
Una fila	`f fijo; for c { m[f][c] }`	Libres en una fila
Una columna	`c fijo; for f { m[f][c] }`	Ocupados en una columna
Diagonal principal	`for i { m[i][i] }`	Solo en matrices cuadradas
Diagonal secundaria	`for i { m[i][n-1-i] }`	Solo en matrices cuadradas
Vecinos de (f, c)	`for df en -1..1, dc en -1..1`	Validar que el vecino exista

Un error frecuente es invertir los índices: `m[c][f]` en lugar de `m[f][c]`. En una matriz cuadrada el programa funciona pero da resultados equivocados; en una rectangular se sale de los límites.

2.3 Limitación: el tamaño fijo

Para pasar una matriz automática a una función hay que indicar la cantidad de columnas en el parámetro:

```
1  void mostrar(int m[][4], int filas) {      // la cantidad de columnas es fija
2      for (int f = 0; f < filas; f++) {
3          for (int c = 0; c < 4; c++)
4              cout << m[f][c] << " ";
5          cout << endl;
6      }
7  }
```

Esa función solo sirve para matrices de 4 columnas. Cuando las dimensiones se conocen en tiempo de ejecución (una sala de tamaño variable, por ejemplo), se necesita una matriz dinámica.

3. Matrices dinámicas

3.1 Matriz dinámica de tipos primitivos

Una matriz dinámica se construye como un **arreglo de filas**: un arreglo de punteros en el que cada puntero apunta a una fila, que a su vez es un arreglo creado con `new`.

```
// Creación
int** m = new int*[filas];          // arreglo de punteros a filas
for (int f = 0; f < filas; f++)
    m[f] = new int[columnas];       // cada fila es un arreglo de enteros

// Uso: igual que una matriz automática
for (int f = 0; f < filas; f++)
    for (int c = 0; c < columnas; c++)
        m[f][c] = f * columnas + c + 1;

// Liberación: en orden inverso a la creación
for (int f = 0; f < filas; f++)
    delete[] m[f];                  // primero cada fila
delete[] m;                         // después el arreglo de filas
```

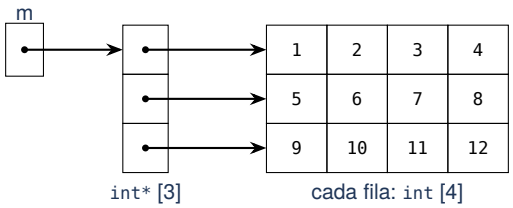


Figura 2. `int**` `m`: un arreglo de 3 punteros, cada uno a una fila de 4 enteros.

Las filas ya no están necesariamente juntas en memoria, pero `m[f][c]` funciona igual: `m[f]` es la fila y `[c]` el elemento dentro de ella. La liberación sigue el orden inverso de la creación: **primero cada fila, después el arreglo de filas**. Si se libera primero el arreglo de filas, se pierden los punteros a las filas y ya no hay forma de liberarlas.

3.2 Matriz dinámica de punteros a objetos

En las colecciones del curso cada celda no guarda un objeto, sino un **puntero** a un objeto. Así una celda puede estar vacía (`nullptr`) y los objetos pueden ser de distintas clases derivadas. Se agrega un nivel de puntero:

Expresión	Tipo	Qué es
`asientos`	`Reserva***`	Puntero al arreglo de filas
`asientos[f]`	`Reserva**`	Una fila: arreglo de punteros a reservas
`asientos[f][c]`	`Reserva*`	Un asiento: apunta a una reserva o es `nullptr`
`asientos[f][c]->getNombre()`	`string`	Llamada a un método del objeto

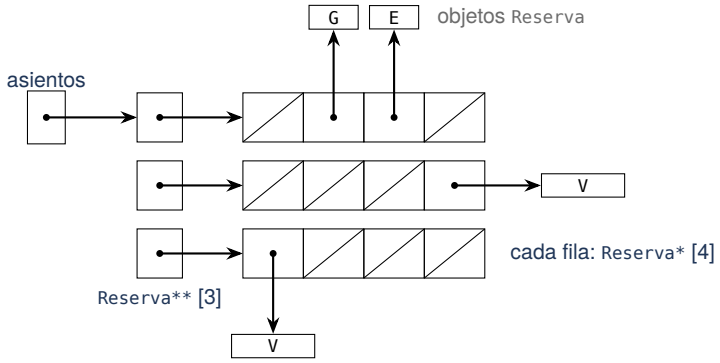


Figura 3. Matriz `Reserva***`: tres niveles. Las diagonales son asientos libres (**`nullptr`**).

```
SalaCine::SalaCine(int filas, int columnas) : filas(filas), columnas(columnas) {
    asientos = new Reserva**[filas];              // 1. arreglo de filas
    for (int f = 0; f < filas; f++) {
        asientos[f] = new Reserva*[columnas];     // 2. cada fila
        for (int c = 0; c < columnas; c++)
            asientos[f][c] = nullptr;             // 3. asiento libre
    }
}
```

El paso 3 no es opcional. Un arreglo creado con **`new`** contiene basura; sin inicializar en **`nullptr`**, el programa no puede distinguir un asiento libre de uno ocupado y el destructor intentaría liberar direcciones al azar.

```
SalaCine::~SalaCine() {
    for (int f = 0; f < filas; f++) {
        for (int c = 0; c < columnas; c++)
            delete asientos[f][c];                // 1. objetos
        delete[] asientos[f];                     // 2. cada fila
    }
    delete[] asientos;                            // 3. arreglo de filas
}
```

La liberación tiene ahora **tres** pasos, en orden inverso a la creación: los objetos, cada fila y el arreglo de filas. Hacer **`delete`** sobre **`nullptr`** es válido y no hace nada, por eso no hace falta preguntar si el asiento está ocupado antes de liberarlo.

> **Alternativa.** También se puede representar la matriz con un solo arreglo de $filas \times columnas$ punteros y calcular la posición como `f * columnas + c`. Es más simple de crear y liberar, pero menos legible. En el curso se usa la versión de arreglo de filas.

4. La matriz como colección encapsulada

La matriz es un detalle de implementación. La clase `SalaCine` no la expone: no tiene un método `getAsientos()`. En cambio, ofrece operaciones con sentido para el problema. Es la Ley de Deméter de la semana 13: quien usa la sala le pide que reserve o cuente asientos, y no manipula la matriz por su cuenta.

```
class SalaCine {
private:
    Reserva*** asientos;     // matriz dinámica de punteros a Reserva
    int filas;
    int columnas;
public:
    SalaCine(int filas, int columnas);
    ~SalaCine();
    bool posicionValida(int f, int c) const;
    bool reservar(int f, int c, Reserva* r);
    bool cancelar(int f, int c);
    int libresEnFila(int f) const;
    int ocupadosEnColumna(int c) const;
    int filaMasOcupada() const;
    bool buscarPorCedula(string cedula, int& f, int& c) const;
    bool mover(int f1, int c1, int f2, int c2);
    double recaudacion() const;
    int contarVIPConCombo() const;
    int estudiantesEnFila(int f) const;
    string mapa() const;
};
```

4.1 Validar antes de acceder

Toda operación que recibe una posición la valida primero. Un índice fuera de rango no produce un error de compilación: lee o escribe memoria ajena.

```
bool SalaCine::posicionValida(int f, int c) const {
    return f >= 0 && f < filas && c >= 0 && c < columnas;
}
```

```
// Si retorna false, la reserva NO quedó en la sala: quien llama debe liberarla.
bool SalaCine::reservar(int f, int c, Reserva* r) {
    if (!posicionValida(f, c) || asientos[f][c] != nullptr)
        return false;
    asientos[f][c] = r;
    return true;
}
```

`reservar` recibe un objeto creado con **`new`** por quien llama. Si la reserva se acepta, la sala pasa a ser responsable de liberarla; si se rechaza, quien llamó debe liberarla. Esa regla debe quedar documen-

tada.

```
bool SalaCine::cancelar(int f, int c) {
    if (!posicionValida(f, c) || asientos[f][c] == nullptr)
        return false;
    delete asientos[f][c];
    asientos[f][c] = nullptr;                    // el asiento vuelve a estar libre
    return true;
}
```

Después del **`delete`** hay que dejar la celda en **`nullptr`**. Si no, la celda queda con la dirección de un objeto que ya no existe: el asiento parece ocupado y el destructor lo liberaría por segunda vez.

4.2 Recorridos con sentido

```
int SalaCine::libresEnFila(int f) const {
    if (f < 0 || f >= filas)
        return -1;
    int libres = 0;
    for (int c = 0; c < columnas; c++)           // fila fija, se mueve la columna
        if (asientos[f][c] == nullptr)
            libres++;
    return libres;
}

int SalaCine::ocupadosEnColumna(int c) const {
    if (c < 0 || c >= columnas)
        return -1;
    int ocupados = 0;
    for (int f = 0; f < filas; f++)              // columna fija, se mueve la fila
        if (asientos[f][c] != nullptr)
            ocupados++;
    return ocupados;
}

int SalaCine::filaMasOcupada() const {
    int mejor = -1, maximo = 0;
    for (int f = 0; f < filas; f++) {
        int ocupados = columnas - libresEnFila(f);
        if (ocupados > maximo) {
            maximo = ocupados;
            mejor = f;
        }
    }
    return mejor;                                // -1 si la sala está vacía
}
```

Observe que `filaMasOcupada` reutiliza `libresEnFila` en lugar de repetir el ciclo.

4.3 Retornar una posición: parámetros por referencia

Una búsqueda en matriz debe informar dos números, la fila y la columna, y además si encontró algo. Una solución común es retornar **bool** y entregar la posición mediante parámetros por referencia:

```
bool SalaCine::buscarPorCedula(string cedula, int& f, int& c) const {
    for (int i = 0; i < filas; i++)
        for (int j = 0; j < columnas; j++)
            if (asientos[i][j] != nullptr && asientos[i][j]->getCedula() == cedula) {
                f = i;                          // se "retorna" la posición
                c = j;                          // por medio de las referencias
                return true;
            }
    return false;
}
```

```
int f, c;
if (sala.buscarPorCedula("5-5555-5555", f, c))
    cout << "Pedro esta en (" << f << "," << c << ")" << endl;
```

4.4 Mover sin copiar

```
bool SalaCine::mover(int f1, int c1, int f2, int c2) {
    if (!posicionValida(f1, c1) || !posicionValida(f2, c2))
        return false;
    if (asientos[f1][c1] == nullptr || asientos[f2][c2] != nullptr)
        return false;
    asientos[f2][c2] = asientos[f1][c1];          // se mueve el puntero, no el objeto
    asientos[f1][c1] = nullptr;
    return true;
}
```

Mover un objeto dentro de la matriz no requiere **new** ni **delete**: basta con copiar el *puntero* al destino y dejar el origen en **nullptr**. El objeto sigue siendo el mismo.

5. Herencia dentro de la colección

Las celdas guardan punteros a la clase base `Reserva`, así que cada una puede apuntar a cualquier clase derivada:

```cpp
// Clase base abstracta: toda reserva tiene un cliente y un precio.
class Reserva {
protected:
    string cedula;
    string nombre;
public:
    Reserva(string cedula, string nombre) : cedula(cedula), nombre(nombre) {}
    virtual ~Reserva() {}
    string getCedula() const { return cedula; }
    string getNombre() const { return nombre; }
    virtual double calcularPrecio() const = 0;
    virtual char getSimbolo() const = 0;     // letra para el mapa de la sala
};
```

```cpp
class ReservaGeneral : public Reserva {
public:
    ReservaGeneral(string cedula, string nombre) : Reserva(cedula, nombre) {}
    double calcularPrecio() const override { return 3500; }
    char getSimbolo() const override { return 'G'; }
};

class ReservaEstudiante : public Reserva {
private:
    string carne;
public:
    ReservaEstudiante(string cedula, string nombre, string carne)
        : Reserva(cedula, nombre), carne(carne) {}
    string getCarne() const { return carne; }
    double calcularPrecio() const override { return 3500 * 0.6; }   // 40 % menos
    char getSimbolo() const override { return 'E'; }
};

class ReservaVIP : public Reserva {
private:
    bool combo;
public:
    ReservaVIP(string cedula, string nombre, bool combo)
        : Reserva(cedula, nombre), combo(combo) {}
    bool tieneCombo() const { return combo; }
    double calcularPrecio() const override { return 6500 + (combo ? 2500 : 0); }
    char getSimbolo() const override { return 'V'; }
};
```

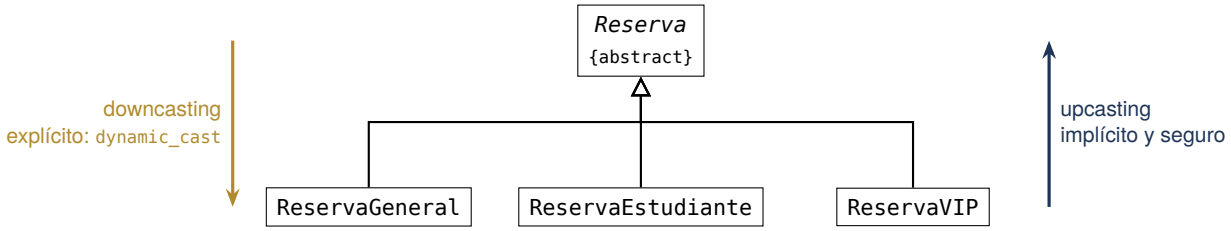


Figura 4. Jerarquía de reservas. Subir en la jerarquía es automático; bajar requiere verificación.

5.1 Upcasting: de derivada a base

Al reservar se pasa un `ReservaVIP*` donde se espera un `Reserva*`. La conversión es implícita porque siempre es segura: toda reserva VIP *es una* reserva.

```
sala.reservar(1, 4, new ReservaVIP("4-4444-4444", "Marta", true));   // upcasting
```

5.2 Polimorfismo: no hace falta saber el tipo

Para calcular la recaudación, la sala le pide a cada reserva su precio. El enlace dinámico ejecuta la versión de la clase real de cada objeto:

```
1 double SalaCine::recaudacion() const {
2     double total = 0;
3     for (int f = 0; f < filas; f++)
4         for (int c = 0; c < columnas; c++)
5             if (asientos[f][c] != nullptr)
6                 total += asientos[f][c]->calcularPrecio();   // enlace dinámico
7     return total;
8 }
```

Lo mismo hace `mapa()` con `getSimbolo()`. Ninguno de los dos métodos pregunta de qué tipo es cada reserva, y ninguno cambia si mañana se agrega una reserva de adulto mayor.

6. Downcasting

6.1 ¿Cuándo se necesita?

A través de un `Reserva*` solo se pueden invocar los métodos declarados en `Reserva`. Si se necesita algo que solo existe en una derivada, como saber si una reserva VIP incluye combo, hay que convertir el puntero a la clase derivada. Esa conversión hacia abajo en la jerarquía se llama **downcasting**, y a diferencia del upcasting **no siempre es válida**: un `Reserva*` puede apuntar a una reserva general, que no tiene combo.

```
Reserva* r = asientos[f][c];
r->tieneCombo();                          // NO compila: Reserva no lo declara
```

6.2 `dynamic_cast` y `static_cast`

	`dynamic_cast<Derivada*>(p)`	`static_cast<Derivada*>(p)`
¿Verifica el tipo?	Sí, en tiempo de ejecución.	No: confía en el programador.
Si el objeto no es de ese tipo	Retorna `nullptr`.	Retorna un puntero inválido; usarlo es comportamiento indefinido.
Si `p` es `nullptr`	Retorna `nullptr`.	Retorna `nullptr`.
Requisito	La base debe tener al menos un método virtual.	Ninguno.
Cuándo usarlo	Siempre que no se tenga certeza del tipo real.	Solo si el tipo está garantizado por otra vía.

En las colecciones del curso casi nunca se tiene certeza del tipo de cada celda, así que la herramienta correcta es `dynamic_cast`.

6.3 El patrón: convertir, verificar, usar

```
int SalaCine::contarVIPConCombo() const {
    int n = 0;
    for (int f = 0; f < filas; f++)
        for (int c = 0; c < columnas; c++) {
            ReservaVIP* v = dynamic_cast<ReservaVIP*>(asientos[f][c]);
            if (v != nullptr && v->tieneCombo())
                n++;
        }
    return n;
}

int SalaCine::estudiantesEnFila(int f) const {
    if (f < 0 || f >= filas)
        return -1;
    int n = 0;
    for (int c = 0; c < columnas; c++)
        if (dynamic_cast<ReservaEstudiante*>(asientos[f][c]) != nullptr)
            n++;
    return n;
}
```

Tres detalles:

- **Siempre se verifica el resultado** antes de usarlo. Si el asiento tiene una reserva general, `v` es `nullptr` y `v->tieneCombo()` haría caer el programa.
- **No hace falta preguntar si la celda está vacía.** Aplicado a `nullptr`, `dynamic_cast` retorna `nullptr`, de modo que la misma verificación cubre los asientos libres.
- En `estudiantesEnFila` solo interesa *si* la conversión funciona, no usar el puntero resultante.

6.4 Con referencias: excepción `bad_cast`

`dynamic_cast` también funciona con referencias. Como una referencia no puede ser nula, cuando el objeto no es del tipo pedido se lanza la excepción `std::bad_cast` (encabezado `<typeinfo>`):

```
try {
    ReservaVIP& vip = dynamic_cast<ReservaVIP&>(*asientos[f][c]);
    cout << vip.tieneCombo() << endl;
} catch (const bad_cast& e) {
    cout << "La reserva no es VIP" << endl;
}
```

En el curso es más común la versión con punteros, porque comparar con **`nullptr`** es más simple que manejar excepciones.

6.5 "Es un", no "es exactamente"

`dynamic_cast` responde si el objeto *se puede tratar como* el tipo pedido. Por ejemplo, si existiera una clase `ReservaPremium` que heredara de `ReservaVIP`, la conversión a `ReservaVIP*` también tendría éxito con un objeto premium, porque una reserva premium *es una* reserva VIP.

6.6 Usarlo con moderación

El downcasting es apropiado cuando la operación *solo tiene sentido* para algunos tipos (contar combos, listar carnés de estudiante). Si se usa para decidir cómo calcular algo que todos los tipos tienen, es señal de que falta un método virtual:

```
// Diseño a evitar: el precio debería ser un método virtual
if (dynamic_cast<ReservaVIP*>(r) != nullptr) total += 6500;
else if (dynamic_cast<ReservaEstudiante*>(r) != nullptr) total += 2100;
else total += 3500;
```

7. Errores frecuentes

Error	Consecuencia	Corrección
Invertir los índices (`m[c][f]`)	Resultados incorrectos o acceso fuera de los límites	Convención fija: primero fila, después columna.
No inicializar las celdas en `nullptr`	Celdas con basura: parecen ocupadas y el destructor libera direcciones al azar	Inicializar en el constructor.
Liberar el arreglo de filas antes que las filas	Las filas quedan inaccesibles: fuga de memoria (y uso de memoria liberada)	Orden inverso a la creación.
Usar `delete` en lugar de `delete[]` con un arreglo	Comportamiento indefinido	`delete[]` para todo lo creado con `new[]`.
Liberar solo las filas y olvidar los objetos	Fuga de todos los objetos	Tres niveles: objetos, filas, arreglo.
No dejar `nullptr` después de liberar una celda	Doble liberación en el destructor	`delete p; p = nullptr;`
No validar la posición	Lectura o escritura de memoria ajena	Validar al inicio de cada método.
Usar el resultado de `dynamic_cast` sin verificarlo	Desreferenciar `nullptr`	Comparar con `nullptr`.
Usar `static_cast` para bajar sin certeza del tipo	Puntero inválido, fallos difíciles de rastrear	Usar `dynamic_cast`.
Destructor de la base sin `virtual`	No se ejecuta el destructor de la derivada	`virtual ~Base() {}`.

8. Resumen

Concepto	Sintaxis	Para qué
Matriz automática	`int m[3][4];`	Dimensiones fijas al compilar.
Matriz dinámica	`int** m = new int*[F];` y `m[f] = new int[C];`	Dimensiones en ejecución.
Matriz de objetos	`Clase*** m;` con celdas en `nullptr`	Colección con espacios vacíos.
Liberación	objetos, `delete[] m[f]`, `delete[] m`	Sin fugas.
Posición por referencia	`bool buscar(..., int& f, int& c)`	Retornar dos valores.
Upcasting	`Base* p = new Derivada(...);`	Guardar cualquier derivada.
Downcasting verificado	`Derivada* d = dynamic_cast<Derivada*>(p);`	Usar lo específico de un tipo.

Referencias

Deitel, H. y Deitel, P. (2016). *Cómo programar en C++* (11.ª ed.). Prentice Hall.

Joyanes, L. (1996). *Programación orientada a objetos: conceptos, modelado, diseño y codificación en C++*. McGraw-Hill.

A. Programa de prueba

El código completo está en `sala_cine.cpp`. Compila con `g++ -std=c++11` o superior y fue verificado sin fugas de memoria. Su función `main`:

```
int main() {
    cout << fixed << setprecision(0) << boolalpha;
    SalaCine sala(4, 6);
    sala.reservar(0, 1, new ReservaGeneral("1-1111-1111", "Ana"));
    sala.reservar(0, 2, new ReservaEstudiante("2-2222-2222", "Luis", "B12345"));
    sala.reservar(0, 3, new ReservaEstudiante("3-3333-3333", "Sofia", "B23456"));
    sala.reservar(1, 4, new ReservaVIP("4-4444-4444", "Marta", true));
    sala.reservar(2, 0, new ReservaVIP("5-5555-5555", "Pedro", false));
    sala.reservar(2, 5, new ReservaGeneral("6-6666-6666", "Carlos"));
    sala.reservar(3, 2, new ReservaVIP("7-7777-7777", "Elena", true));
    cout << sala.mapa();

    Reserva* r = new ReservaGeneral("8-8888-8888", "Jose");
    bool ok = sala.reservar(0, 2, r);              // asiento ocupado
    cout << "Reservar (0,2) ocupado: " << ok << endl;
    if (!ok)
        delete r;
    r = new ReservaGeneral("8-8888-8888", "Jose");
    ok = sala.reservar(9, 9, r);                   // fuera de rango
    cout << "Reservar (9,9) invalido: " << ok << endl;
    if (!ok)
        delete r;

    cout << "Libres en fila 0: " << sala.libresEnFila(0) << endl;
    cout << "Ocupados en columna 2: " << sala.ocupadosEnColumna(2) << endl;
    cout << "Fila mas ocupada: " << sala.filaMasOcupada() << endl;
    cout << "Recaudacion: " << sala.recaudacion() << endl;
    cout << "VIP con combo: " << sala.contarVIPConCombo() << endl;
    cout << "Estudiantes en fila 0: " << sala.estudiantesEnFila(0) << endl;

    int f, c;
    if (sala.buscarPorCedula("5-5555-5555", f, c))
        cout << "Pedro esta en (" << f << "," << c << ")" << endl;
    cout << "Mover (2,0) a (3,5): " << sala.mover(2, 0, 3, 5) << endl;
    cout << "Cancelar (0,1): " << sala.cancelar(0, 1) << endl;
    cout << "Cancelar (0,1) otra vez: " << sala.cancelar(0, 1) << endl;
    cout << sala.mapa();
```

```
38      cout << "Recaudacion: " << sala.recaudacion() << endl;
39      return 0;
40  }
```

Salida del programa:

```
Fila 0: . G E E . .
Fila 1: . . . . V .
Fila 2: V . . . . G
Fila 3: . . V . . .
Reservar (0,2) ocupado: false
Reservar (9,9) invalido: false
Libres en fila 0: 3
Ocupados en columna 2: 2
Fila mas ocupada: 0
Recaudacion: 35700
VIP con combo: 2
Estudiantes en fila 0: 2
Pedro esta en (2,0)
Mover (2,0) a (3,5): true
Cancelar (0,1): true
Cancelar (0,1) otra vez: false
Fila 0: . . E E . .
Fila 1: . . . . V .
Fila 2: . . . . . G
Fila 3: . . V . . V
Recaudacion: 32200
```